

Motorsport Analytics Platform

Comprehensive Project Proposal

Executive Summary

This project proposes a full-stack motorsport analytics platform powered by OpenF1 data. The system will provide live timing, telemetry analysis, driver comparisons, race strategy tools, and AI-powered insights through a Java Spring Boot backend and React frontend.

Business Objectives

Deliver a modern motorsport analytics experience, demonstrate enterprise software engineering skills, and provide a scalable foundation for future motorsport datasets.

Stakeholders

Fans, analysts, developers, project assessors, and potential commercial users.

System Architecture

OpenF1 API -> Spring Boot Backend -> Business Services -> REST APIs -> React Frontend.
Optional PostgreSQL and Redis support persistence and caching.

Technology Stack

Java 21, Spring Boot, WebClient, PostgreSQL, Redis, React, TypeScript, Axios, React Query, Chart.js/Recharts, Docker, GitHub Actions.

User Stories

Users can view live sessions, compare drivers, inspect telemetry, review race timelines, and receive AI-generated race summaries.

Functional Requirements

Driver management, session retrieval, lap analysis, telemetry visualization, race strategy analysis, team radio exploration, championship prediction, AI insight generation.

Non-Functional Requirements

Performance, scalability, maintainability, availability, security, usability, accessibility, and observability.

API Design

REST endpoints for drivers, sessions, telemetry, laps, positions, weather, and strategy analysis.

Database Design

Driver, Session, Lap, Telemetry, RaceEvent, UserPreferences, and Analytics entities.

Frontend Design

Dashboard, Driver Comparison, Telemetry Explorer, Strategy Center, Championship Predictor, and Settings modules.

Backend Design

Controller, Service, Repository, DTO, and Client layers following clean architecture principles.

Security

Input validation, rate limiting, HTTPS, secret management, and secure deployment practices.

Testing Strategy

Unit testing, integration testing, API testing, UI testing, and performance testing.

CI/CD Pipeline

GitHub Actions for build, test, containerization, and deployment.

Deployment Architecture

Containerized services deployed to cloud infrastructure with monitoring and logging.

Risk Analysis

External API availability, data quality, scaling challenges, and operational risks.

Project Roadmap

Phase 1: Core APIs. Phase 2: Telemetry. Phase 3: Strategy analytics. Phase 4: AI insights.

Future Enhancements

Fantasy motorsport, machine learning predictions, mobile apps, notifications, and multi-series support.

Suggested API Endpoints

GET /api/sessions

GET /api/drivers/{sessionKey}

GET /api/laps/{sessionKey}

GET /api/telemetry/{driverNumber}

GET /api/strategy/{sessionKey}